UFRGS
UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL

IA PARA ENGENHEIROS
UFRGS

Curso de Especialização

Inteligência Artificial para Engenheiros

Disciplina: **Algoritmos para Agrupamento de Dados**

Aula: **Métricas para Avaliação de Desempenho**

Professor: **Maurício Cagliari Tosin**

2026

Aprendizagem Supervisionada x Não Supervisionada

Aprendizagem não supervisionada versus supervisionada

- Aprendizagem Não Supervisionada:
  - área que busca encontrar padrões em dados desordenados;
- Aprendizagem Supervisionada:
  - problemas representados por conjuntos de dados rotulados;
  - metodologia muito usada para classificar dados com base em rótulos:
    - por exemplo, classificar se uma pessoa é fumante ou não fumante com base na análise dos dados sobre saúde pulmonar;
  - ou para encontrar um padrão claramente definido nos dados:
    - por exemplo, prever o preço de venda de uma casa com base na quantidade de quartos;
  - este modelo espelha de forma mais precisa a abordagem humana intuitiva para a aprendizagem;
    - por exemplo aprender como não queimar uma comida em função do conhecimento básico de culinária:
      - você pode construir um conjunto de dados colocando sua comida no fogão e verificando quanto tempo leva (entrada) para ela queimar (saída);
      - à medida que continua a queimar sua comida, você construirá um modelo mental de quando ocorrerá a queima e como evitá-la no futuro.

Aprendizagem não supervisionada versus supervisionada

APRENDIZAGEM	
Não Supervisionada	**Supervisionada**
Nenhum rótulo é necessário	Rótulos são necessários
Encontrar estruturas em dados desbalanceados	Encontrar modelos em estruturas existentes
Uso de técnicas tais como agrupamento de dados (*clustering*) e redução de dimensionalidade	Uso de técnicas tais como regressão e classificação

Avaliação de Desempenho em Aprendizagem Supervisionada

Principais Métricas

- **Matriz de Confusão:** é uma tabela que permite a visualização do desempenho de um algoritmo de classificação.

$VP: verdadeiro\ positivo$

$FN: falso\ negativo$

$FP: falso\ positivo$

$VN: verdadeiro\ positivo$

$p: número\ de\ positivos$

$n: número\ de\ negativos$

$N: número\ de\ amostras$

Matriz de Confusão para 2 classes

	Classe Predita		
Classe verdadeira	Positiva	Negativa	*Total*
Positiva	VP: verdadeiro positivo	FN: falso negativo	p
Negativa	FP: falso positivo	VN: verdadeiro negativo	n
Total	p'	n'	*N*

Principais Métricas

- **Acurácia (*accuracy*):** A proporção de predições corretas, sem levar em consideração o que é positivo e o que é negativo. Esta medida é altamente suscetível a desbalanceamento do conjunto de dados e pode facilmente induzir a uma conclusão errada sobre o desempenho do sistema.

$$Acc = \frac{VP + VN}{N}$$

	Classe Predita		
Classe verdadeira	Positiva	Negativa	*Total*
Positiva	VP: verdadeiro positivo	FN: falso negativo	p
Negativa	FP: falso positivo	VN: verdadeiro negativo	n
Total	p'	n'	*N*

Principais Métricas

- **Sensitividade (*sensitivity*) ou *recall*:** A proporção de verdadeiros positivos (VP): a capacidade do sistema em predizer corretamente a condição para casos que realmente a têm.

$$Sen = Recall = \frac{VP}{VP + FN} = \frac{VP}{p}$$

	Classe Predita		
Classe verdadeira	Positiva	Negativa	*Total*
Positiva	VP: verdadeiro positivo	FN: falso negativo	p
Negativa	FP: falso positivo	VN: verdadeiro negativo	n
Total	p'	n'	*N*

Principais Métricas

- **Specificidade (*specificity*):** A proporção de verdadeiros negativos (VN): a capacidade do sistema em predizer corretamente a ausência da condição para casos que realmente não a têm. Essa métrica é aplicável para problemas de duas classes.

$$Spec = \frac{VN}{VN + FP} = \frac{VN}{n}$$

	Classe Predita		
Classe verdadeira	Positiva	Negativa	*Total*
Positiva	VP: verdadeiro positivo	FN: falso negativo	p
Negativa	FP: falso positivo	VN: verdadeiro negativo	n
Total	p'	n'	*N*

Principais Métricas

- **Eficiência:** A média aritmética da Sensibilidade (*Recall*) e Especificidade. Na prática, a sensibilidade e a especificidades variam em direções opostas, isto é, geralmente, quando um método é muito sensível a positivos, tende a gerar muitos falsos-positivos, e vice-versa. Assim, um método de decisão perfeito (100% de sensibilidade e 100% de especificidade) realmente é alcançado, e um balanço entre ambos deve ser atingido. Aplicável para problemas de duas classes.

$$Efic = \frac{Recall + Spec}{2}$$

	Classe Predita		
Classe verdadeira	Positiva	Negativa	*Total*
Positiva	VP: verdadeiro positivo	FN: falso negativo	p
Negativa	FP: falso positivo	VN: verdadeiro negativo	n
Total	p'	n'	*N*

Principais Métricas

- **Precisão:** A proporção de verdadeiros positivos em relação a todas as predições positivas. Esta medida é altamente suscetível a desbalanceamentos do conjunto de dados e pode facilmente induzir a uma conclusão errada sobre o desempenho do sistema.

$$Prec = \frac{VP}{VP + FP} = \frac{VP}{p'}$$

	Classe Predita		
Classe verdadeira	Positiva	Negativa	*Total*
Positiva	VP: verdadeiro positivo	FN: falso negativo	p
Negativa	FP: falso positivo	VN: verdadeiro negativo	n
Total	p'	n'	*N*

Principais Métricas

- **F1-Score:** É a média harmônica de precisão e *recall*.

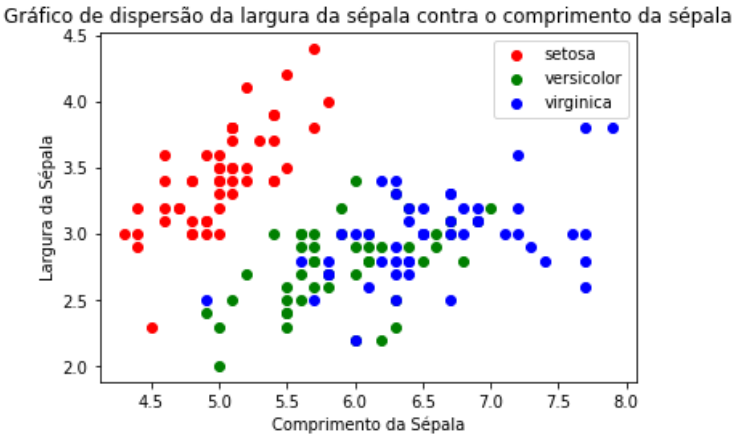
$$F1\ score = 2 \cdot \frac{Precisão \cdot Recall}{Precisão + Recall}$$

	Classe Predita		
Classe verdadeira	Positiva	Negativa	*Total*
Positiva	VP: verdadeiro positivo	FN: falso negativo	p
Negativa	FP: falso positivo	VN: verdadeiro negativo	n
Total	p'	n'	*N*

Principais Métricas – Aprendizado supervisionado

- **Exemplo 01:** Vamos implementar o método k-NN no conjunto de dados da Flor de Iris. Para tanto, vamos considerar as características de Largura da Sépala e Comprimento da Sépala para classificar três espécies: Setosa, Versicolor e Virgínica.

```
import pandas as pd
import numpy as np
import matplotlib.patches as mpatches
from sklearn import svm, datasets
import matplotlib.pyplot as plt
iris = datasets.load_iris()
X = iris.data[:, :2] #usar as primeiras duas características (features)
y = iris.target
#plotar os dados
colors = ['red', 'green', 'blue']
for color, i, target in zip(colors, [0, 1, 2], iris.target_names):
    plt.scatter(X[y==i, 0], X[y==i, 1], color=color, label=target)
plt.xlabel('Comprimento da Sépala')
plt.ylabel('Largura da Sépala')
plt.legend(loc='best', shadow=False, scatterpoints=1)
plt.title('Gráfico de dispersão da largura da sépala contra o comprimento da sépala')
plt.show()
```

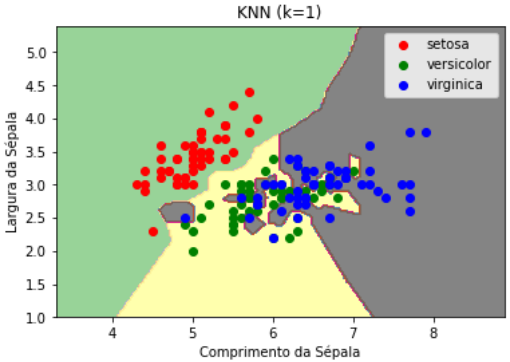


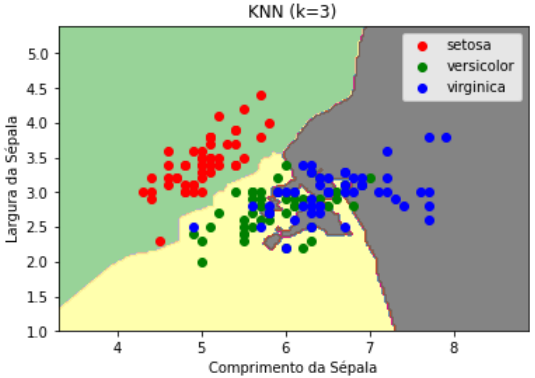
Principais Métricas – Aprendizado supervisionado

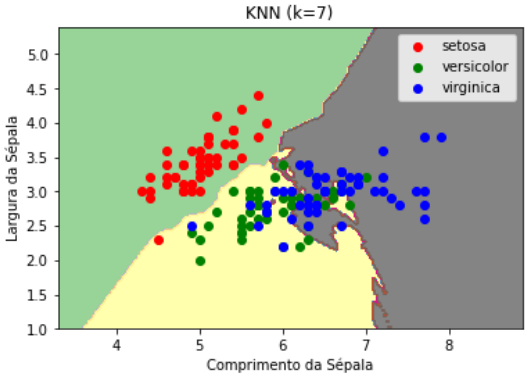
- Explorando diferentes valores de k (iniciamos com k = 1):

```
from sklearn.neighbors import KNeighborsClassifier
k = 1
#instanciar modelo de aprendizagem ---
knn = KNeighborsClassifier(n_neighbors=k)
#---ajustando o modelo ---
knn.fit(X, y)
#mínimo e máximo para o primeira característica
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
#mínimo e máximo para a segunda característica
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
#tamanho do passo
h = (x_max / x_min)/100
#fazer previsões para cada um dos dados em xx,yy
xx, yy = np.meshgrid(np.arange(x_min, x_max, h),np.arange(y_min, y_max, h))
Z = knn.predict(np.c_[xx.ravel(), yy.ravel()])
#desenhe o resultado usando um gráfico de cores
Z = Z.reshape(xx.shape)
plt.contourf(xx, yy, Z, cmap=plt.cm.Accent, alpha=0.8)
#traçar os dados de treinamento
colors = ['red', 'green', 'blue']
for color, i, target in zip(colors, [0, 1, 2], iris.target_names):
    plt.scatter(X[y==i, 0], X[y==i, 1], color=color, label=target)
plt.xlabel('Comprimento da Sépala')
plt.ylabel('Largura da Sépala')
plt.title(f'KNN (k={k})')
plt.legend(loc='best', shadow=False, scatterpoints=1)
predictions = knn.predict(X)
#classificações baseadas em previsões
print(np.unique(predictions, return_counts=True))
```

Principais Métricas – Aprendizado supervisionado

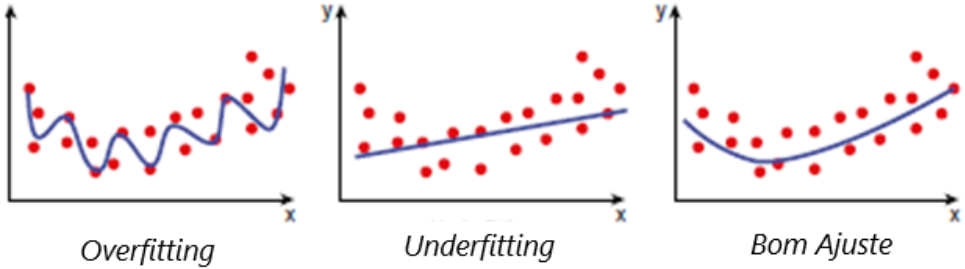






Principais Métricas – Aprendizado supervisionado

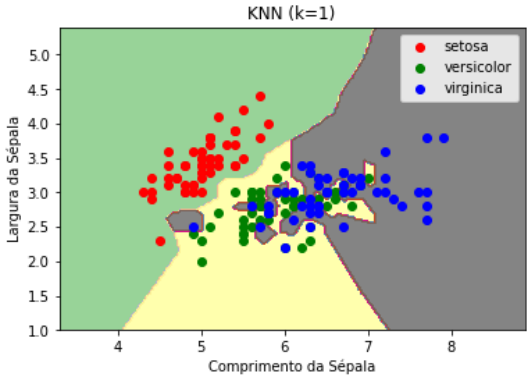
- **Overfitting (sobre-ajuste):** significa que o modelo que você treinou se ajusta muito bem aos dados de treinamento. Porém, o principal problema é que ele não funcionará bem com dados novos (evidentemente não utilizados no treinamento). Portanto, o modelo não generalizou bem para novos dados. Uma maneira de perceber este fenômeno é que o modelo decorou o conjunto de dados de treinamento, mas não aprendeu realmente a diferenciar esses dados para quando precisar enfrentar novos testes (com o conjunto de dados de testes).
- **Underfitting (sub-ajuste):** ocorre quando o modelo proposto não consegue capturar com precisão a tendência subjacente dos dados. Especificamente, o modelo não se ajusta bem aos dados, ou seja, o modelo não se adaptou nem aos dados de treinamento.

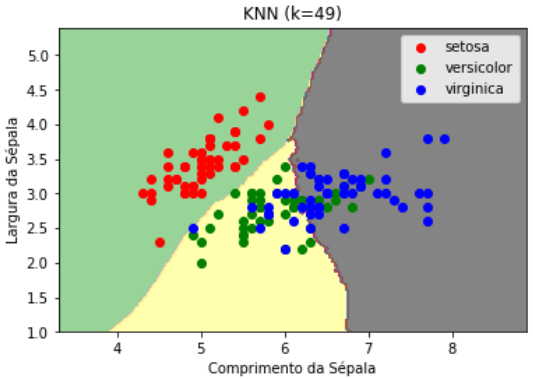


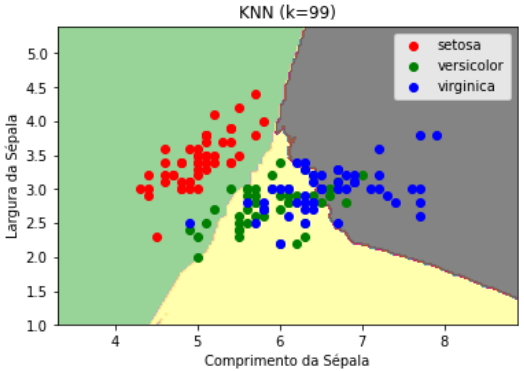
Exemplo genérico comparando o sobre-ajuste, sub-ajuste e um ajuste considerado adequado.

Principais Métricas – Aprendizado supervisionado

- Observar que quando o k aumenta, a divisão (o limite) entre as três classes se torna mais suave. Porém, mais dados foram classificados incorretamente. Quando o k aumenta para um valor alto ocorre um sub-ajuste.
- A questão chave com o algoritmo k-NN é como descobrir o valor ideal de k?





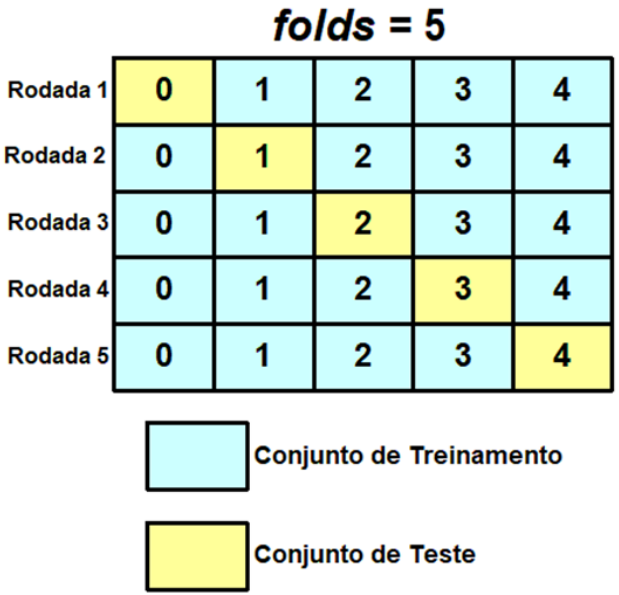


Validação Cruzada

- Normalmente, dividimos a base de dados em treinamento e teste;
- No entanto, os dados em sua totalidade (o dataset) podem não ser distribuídos uniformemente, sendo assim, podendo resultar na formação de um conjunto de teste muito simples ou muito complicado para uma determinada previsão;
- Em vez de usar parte dos dados para treinamento e outra parte para teste, é possível dividir os dados em k-folds e treinar o modelo proposto k vezes, “girando” os dados entre os conjuntos de treinamento e de teste;
- Neste método de validação cruzada possivelmente o dado será usado para treinamento e para teste.

Não confunda o *k* do *k-fold* com o *k* do *k-NN* - não estão relacionados.

Validação Cruzada – k = 5



- Em cada rodada, uma partição é separada para o teste;
- No final de cada execução, do algoritmo de validação cruzada, podemos usar alguma métrica para avaliar o desempenho do seu algoritmo;
- Pode-se tomar a média, por exemplo, das 5 execuções para se estimar uma medida para o desempenho global mais fidedigna;
- O objetivo da validação cruzada não é treinar seu modelo, mas sim verificar o modelo. A validação cruzada é muito útil quando você precisa comparar diferentes algoritmos de Aprendizado de Máquina

Principais Métricas – Validação Cruzada

- **Exemplo 02:** Vamos implementar o método k-NN no conjunto de dados da Flor de Iris. Para tanto, vamos considerar todas as quatro características e, ao mesmo tempo, usaremos a validação cruzada no conjunto de dados usando 10 *folds*.

```
from sklearn.model_selection import cross_val_score
#mantém as pontuações da cv (validação cruzada)
cv_scores = []
#usa todas as características (features)
X = iris.data[:, :4]
y = iris.target
#número de folds
folds = 10
#criando lista ímpar de K para KNN
ks = list(range(1,int(len(X) * ((folds - 1)/folds))))
#remove todos os múltiplos de 3
ks = [k for k in ks if k % 3 != 0]
#realiza a validação cruzada (k-fold)
for k in ks:
    knn = KNeighborsClassifier(n_neighbors=k)
    #realiza a validação cruzada e retorna a "precisão" média
    scores = cross_val_score(knn, X, y, cv=folds, scoring='accuracy')
    mean = scores.mean()
    cv_scores.append(mean)
    print(k, mean)
```

Principais Métricas – Validação Cruzada

A lista dos *ks* conterá os seguintes valores:

```
[1, 2, 4, 5, 7, 8, 10, 11, 13, 14, 16, 17, 19, 20, 22, 23, 25, 26, 28,
29, 31, 32, 34, 35, 37, 38, 40, 41, 43, 44, 46, 47, 49, 50, 52, 53, 55,
56, 58, 59, 61, 62, 64, 65, 67, 68, 70, 71, 73, 74, 76, 77, 79, 80, 82,
83, 85, 86, 88, 89, 91, 92, 94, 95, 97, 98, 100, 101, 103, 104, 106,
107, 109, 110, 112, 113, 115, 116, 118, 119, 121, 122, 124, 125, 127,
128, 130, 131, 133, 134]
```

Após o treinamento as pontuações *cv* conterão uma lista com a métrica acurácia média com base nos diferentes valores de *k*:

```
1 0.96
2 0.9533333333333334
4 0.9666666666666666
5 0.9666666666666668
7 0.9666666666666668
8 0.9666666666666668
10 0.9666666666666668
11 0.9666666666666668
13 0.9800000000000001
14 0.9733333333333334
16 0.9733333333333334
17 0.9733333333333334
19 0.9733333333333334
20 0.9800000000000001
22 0.9666666666666666
23 0.9733333333333334
25 0.9666666666666666
```

...

```
110 0.6533333333333332
112 0.6466666666666665
113 0.6466666666666665
115 0.6466666666666665
116 0.6466666666666665
118 0.6399999999999999
119 0.6399999999999999
121 0.6399999999999999
122 0.6399999999999999
124 0.6399999999999999
125 0.6333333333333332
127 0.6266666666666666
128 0.6199999999999999
130 0.6066666666666667
131 0.5933333333333332
133 0.5666666666666667
134 0.5533333333333333
```

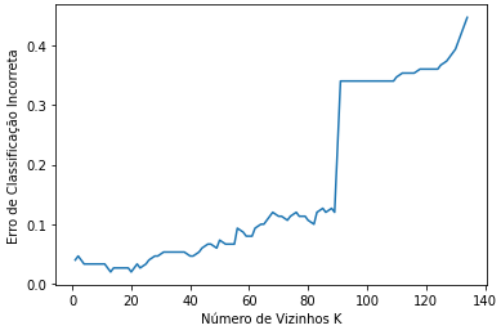
Principais Métricas – Validação Cruzada

- **Exemplo 03:** Para encontrar o k ótimo do algoritmo k-NN podemos tentar encontra o valor de k que forneça o menor erro de classificação incorreta, por exemplo, usando a métrica de erro chamada de MSE (mean squared error). O fragmento de código a seguir encontra o MSE para cada k.

```
#calcular o erro de classificação incorreta para cada k
MSE = [1 - x for x in cv_scores]
#determinando o melhor k (mínimo MSE)
optimal_k = ks[MSE.index(min(MSE))]
print(f"O número de vizinhos é: {optimal_k}")
#plotar o erro de classificação incorreta versus k
plt.plot(ks, MSE)
plt.xlabel('Número de Vizinhos K')
plt.ylabel('Erro de Classificação Incorreta')
plt.show()
```

MSE versus o número de *k*.

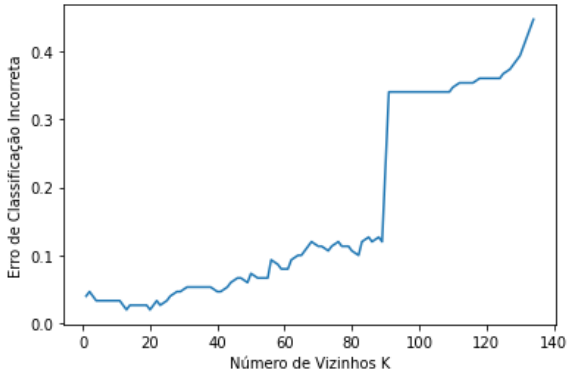
```
O número de vizinhos é: 13
```



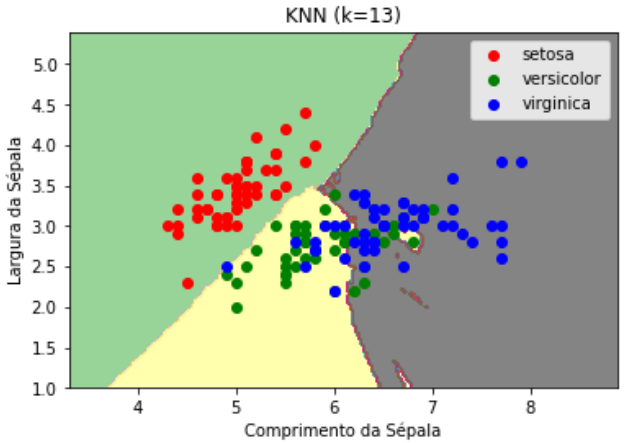
Principais Métricas – Validação Cruzada

MSE versus o número de *k*.

```
O número de vizinhos é: 13
```



A classificação quando *k* = 13.



Métricas para Avaliação de Desempenho de Clusters

Coeficiente de Silhueta

Observações gerais:

- Um procedimento científico bastante usado para avaliar o resultado de um agrupamento é o Coeficiente de Silhueta:
- É uma medida da qualidade do agrupamento determinado;
- Mede a chamada coesão do cluster, ou seja, o espaço entre os clusters;
- Este coeficiente tem uma faixa de valores entre: -1 e 1;
- A expressão mais aceita para sua determinação é 1 – (a / b)
  - a é a distância média de um dado em relação a todos os outros dados de um mesmo cluster:
    - se a for pequeno → a coesão do cluster é boa, pois todos os dados estão próximos;
  - b é a distância média de um dado em relação a todos os outros dados do cluster mais próximo:
    - se b for grande → a separação do cluster é boa, pois o cluster mais próximo está distante;
    - se a for pequeno e b for grande → o Coeficiente de Silhueta é alto;
- O valor de k que produz o maior Coeficiente de Silhueta é considerado o k ideal.

Coeficiente de Silhueta

- **Exemplo 04:** Considere o um conjunto de dados com suas respectivas posições para calcularmos o Coeficiente de Silhueta. Este conjunto tem 7 dados e 3 clusters aos quais os dados pertencem.

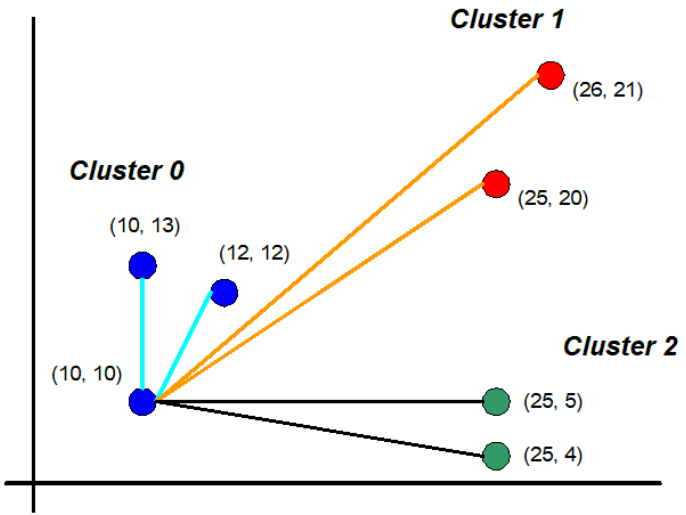
Cálculo do Coeficiente de Silhueta para o dado na posição (10, 10) do *Cluster 0*:

- calcular sua distância média em relação a todos os outros dados do mesmo *Cluster 0* (coeficiente a):

$$(10,10) - (12,12) = \sqrt{8} = 2{,}828$$

$$(10,10) - (10,13) = \sqrt{9} = 3$$

$$média: \frac{(2{,}828 + 3)}{2} = 2{,}914$$



Coeficiente de Silhueta

- calcular sua distância média em relação a todos os outros dados do *Cluster 1*:

$$(10,10) - (25,20) = \sqrt{325} = 18{,}028$$

$$(10,10) - (26,21) = \sqrt{377} = 19{,}416$$

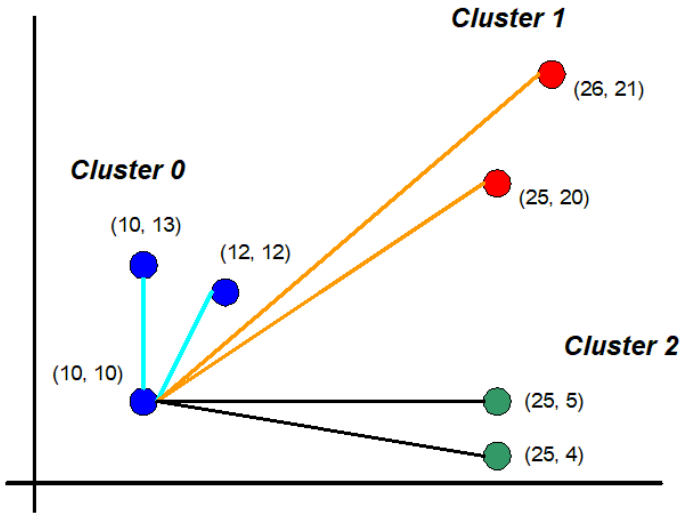
$$média: \frac{(18{,}028 + 19{,}416)}{2} = 18{,}722$$

- calcular sua distância média em relação a todos os outros dados do *Cluster 2*:

$$(10,10) - (25,5) = \sqrt{250} = 15{,}811$$

$$(10,10) - (25,4) = \sqrt{261} = 16{,}155$$

$$média: \frac{(15{,}811 + 16{,}156)}{2} = 15{,}983$$



Coeficiente de Silhueta

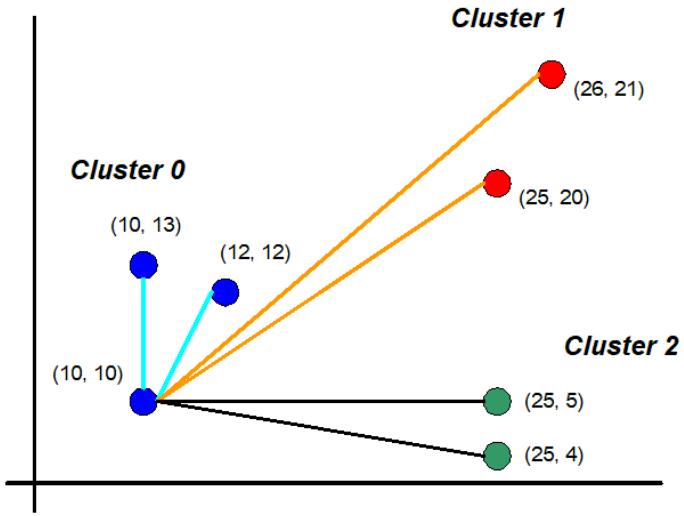
- a distância mínima média do dado (10,10) em relação a todos os *Clusters* 1 e 2:

$$mín(18{,}722; 15{,}983) = 15{,}983$$

- portanto o Coeficiente de Silhueta do dado na posição (10,10) é:

$$1 - \frac{a}{b} = 1 - \frac{2{,}914}{15{,}983} \cong 0{,}818$$

- agora devemos calcular os Coeficientes de Silhueta dos outros seis dados do conjunto exemplo.




```python
# Importa as bibliotecas necessárias
import pandas as pd
from sklearn import metrics

# Gera os dados para o agrupamento
dados = pd.DataFrame({"X":[10,10,12,25,25,25,26], "Y":[10,13,12,4,5,20,21]})
labels = ['Cluster0','Cluster0','Cluster0','Cluster2','Cluster2','Cluster1','Cluster1']

silhouette_samples = metrics.silhouette_samples(dados, labels)
silhouette_score = metrics.silhouette_score(dados, labels)

print("Silhouette Samples: ", silhouette_samples)
print("Silhouette Score: ", silhouette_score)
```

Outras Métricas

- **Exemplo 04:** Vamos implementar o código abaixo para clusterizar uma base de dados gerada contendo 12 amostras bidimensionais. Criaremos 3 clusters pelo método K-means e exploraremos várias métricas para avaliação do desempenho.

```
In [1]: from sklearn import metrics

In [2]: import numpy as np
        import matplotlib.pyplot as plot
        from sklearn.cluster import KMeans
        data_points = np.array([[1, 1], [1, 1.5], [2, 2], \
        [8, 1], [8, 0], [8.5, 1], \
        [6, 1], [1, 10], [1.5, 10], \
        [1.5, 9.5], [10, 10], [1.5, 8.5]])
        k_means_model = KMeans(n_clusters=3,random_state = 8)
        k_means_model.fit(data_points)
        k_means_model.labels_

Out[2]: array([2, 2, 2, 0, 0, 0, 0, 1, 1, 1, 1, 1])

In [3]: data_labels = np.array([0, 0, 0, 2, 2, 2, 2, 1, 1, 1, 3, 1])
```

→ Predições do K-Means

→ Labels verdadeiras

Abordagens Extrínsecas (quando temos a label correta)

O Índice Rand Ajustado

- função que mede a similaridade entre as previsões do cluster e os rótulos ignorando as permutações;
- métrica que funciona muito bem quando os rótulos são grandes grupos de tamanhos iguais;
- esta métrica tem um intervalo entre [-1,1]:
  - valores negativos não são desejáveis;
  - uma pontuação negativa significa que nosso modelo está tendo um desempenho pior do que uma atribuição aleatória dos rótulos;
  - se os atribuíssemos aleatoriamente, nossa pontuação seria próxima de 0;
  - quanto mais próximo esta métrica estiver de 1 melhor será nosso modelo de agrupamento em prever o rótulo correto;

Dados X e Y dois agrupamentos distintos que se queira comparar:

a é o número de pares de elementos que estão no mesmo cluster em X e Y;
b é o número de pares de elementos que estão em diferentes clusters em X e Y;
c é o número de pares de elementos que estão no mesmo cluster em X e em diferentes em Y;
d é o número de pares de elementos que estão no mesmo cluster em Y e em diferentes em X.

Índice de Rand

$$IR = \frac{a+b}{a+b+c+d}$$

Abordagens Extrínsecas (quando temos a label correta)

O Índice Rand Ajustado

- função que mede a similaridade entre as previsões do cluster e os rótulos ignorando as permutações;
- métrica que funciona muito bem quando os rótulos são grandes grupos de tamanhos iguais;
- esta métrica tem um intervalo entre [-1,1]:
  - valores negativos não são desejáveis;
  - uma pontuação negativa significa que nosso modelo está tendo um desempenho pior do que uma atribuição aleatória dos rótulos;
  - se os atribuíssemos aleatoriamente, nossa pontuação seria próxima de 0;
  - quanto mais próximo esta métrica estiver de 1 melhor será nosso modelo de agrupamento em prever o rótulo correto;

E é o valor esperado para o Índice de Rand

Índice de Rand Ajustado

$$IRA = \frac{RI - E}{1 - E}$$

Índice de Rand

$$IR = \frac{a + b}{a + b + c + d}$$

Abordagens Extrínsecas (quando temos a label correta)

O Índice Rand Ajustado

- função que mede a similaridade entre as previsões do cluster e os rótulos ignorando as permutações;
- métrica que funciona muito bem quando os rótulos são grandes grupos de tamanhos iguais;
- esta métrica tem um intervalo entre [-1,1]:
  - valores negativos não são desejáveis;
  - uma pontuação negativa significa que nosso modelo está tendo um desempenho pior do que uma atribuição aleatória dos rótulos;
  - se os atribuíssemos aleatoriamente, nossa pontuação seria próxima de 0;
  - quanto mais próximo esta métrica estiver de 1 melhor será nosso modelo de agrupamento em prever o rótulo correto;
- com a biblioteca sklearn é fácil calcular o Índice Rand Ajustado:

```
In [4]: metrics.adjusted_rand_score(data_labels, k_means_model.labels_)

Out[4]: 0.8422939068100358
```

Abordagens Extrínsecas (quando temos a label correta)

Informações Mútuas Ajustadas

- função que mede a entropia entre as previsões do cluster e os rótulos ignorando as permutações;
- essa métrica não têm intervalo definido:
  - mas os valores negativos são considerados ruins;
  - se os atribuíssemos aleatoriamente, nossa pontuação seria próxima de 0;
  - quanto mais perto esta métrica de desempenho estiver de 1 melhor será o nosso modelo de agrupamento em prever o rótulo certo;

Dados U e V dois agrupamentos distintos que se queira comparar:

IM é a informação mútua entre eles;
E é o valor esperado para a informação mútua;
H é a entropia.

Informações Mútuas Ajustadas

$$IMA(U,V) = \frac{IM(U,V) - E\{IM(U,V)\}}{\max\{H(U),H(V)\} - E\{IM(U,V)\}}$$

Abordagens Extrínsecas (quando temos a label correta)

Informações Mútuas Ajustadas

- função que mede a entropia entre as previsões do cluster e os rótulos ignorando as permutações;
- essa métrica não têm intervalo definido:
  - mas os valores negativos são considerados ruins;
  - se os atribuíssemos aleatoriamente, nossa pontuação seria próxima de 0;
  - quanto mais perto esta métrica de desempenho estiver de 1 melhor será o nosso modelo de agrupamento em prever o rótulo certo;
- com a biblioteca sklearn é fácil calcular a métrica de Informações Mútuas Ajustadas:

```
In [5]: metrics.adjusted_mutual_info_score(data_labels,k_means_model.labels_)

Out[5]: 0.8769185235006342
```

Nesse caso, as informações mútuas ajustadas indicam que nosso modelo k-means é muito bom e não está longe de nossos rótulos verdadeiros.

Abordagens Extrínsecas (quando temos a label correta)

Medida V, Homogeneidade e Completude

- a Medida V é definida como a média harmônica da homogeneidade e completude;
- para calcular a Medida V primeiro precisamos definir homogeneidade e completude (alguns chamam de integridade);
- a **homogeneidade** perfeita se refere a uma situação em que cada cluster possui dados pertencentes ao mesmo rótulo;
  - a pontuação de homogeneidade refletirá o quão bem cada um de nossos clusters está agrupando dados de mesmo rótulo;
- a **completude** perfeita se refere à situação em que todos os dados pertencentes ao mesmo rótulo estão agrupados no mesmo cluster;
- a expressão da Medida V é a seguinte:

Tem valor padrão de 1, mas por se alterado dependendo da ênfase dada a homogeneidade e completude

$$V = \frac{(1+\beta) \times homogeneidade \times completude}{\beta \times homogeneidade + completude}$$

- essas três métricas tem faixa entre [0, 1] onde 0 representa o pior caso possível e 1 o escore perfeito.

Abordagens Extrínsecas (quando temos a label correta)

- com a biblioteca sklearn podemos calcular esta métrica:

```
In [8]: metrics.homogeneity_score(data_labels, k_means_model.labels_)
Out[8]: 0.8378758055108827

In [9]: metrics.completeness_score(data_labels, k_means_model.labels_)
Out[9]: 1.0

In [10]: metrics.v_measure_score(data_labels, k_means_model.labels_,beta=1)
Out[10]: 0.9117871871412709
```

Nesse caso, a métrica homogeneidade indica que nosso algoritmo *k-means* possui *clusters* contendo rótulos diferentes.

Nesse caso, a métrica completude indica que nosso algoritmo *k-means* colocou com sucesso todos os dados de um mesmo rótulo dentro do mesmo *cluster*.

Nesse caso, a Medida V indica que nosso algoritmo *k-means*, embora não seja perfeito, tem um bom desempenho em geral.

Abordagens Extrínsecas (quando temos a label correta)

A Medida de Fowlkes-Mallows

- métrica que mede a similaridade dentro de um cluster rotulado e a previsão deste cluster;
- definida como a média geométrica da precision e recall:

$$FM = FMI = \frac{TP}{\sqrt{(TP + FP)(TP + FN)}}$$

- TP (True Positive): são todas as observações onde as predições estão no mesmo cluster como no cluster rotulado
- FP (False Positive): são todas as observações onde as predições estão no mesmo cluster, mas não estão no mesmo cluster rotulado;
- FN (False Negative): são todas as observações onde as predições não estão no mesmo cluster, mas estão no mesmo cluster rotulado;
- esta métrica tem faixa entre [0, 1] onde 0 é pior escore possível e 1 o escore perfeito;
- na biblioteca sklearn podemos calcular facilmente esta métrica:

```
In [11]: metrics.fowlkes_mallows_score(data_labels, k_means_model.labels_)

Out[11]: 0.8885233166386386
```

Abordagens Intrínsecas (quando não temos as labels)

Métodos usados quando não temos o rótulo:

- o coeficiente de silhueta;
- o índice Calinski-Harabasz;
- o índice Davies-Bouldin.

Abordagens Intrínsecas (quando não temos as labels)

O Índice Calinski-Harabasz:

- mede como os dados dentro de cada cluster estão distribuídos;
- é definido como a razão da variância entre os clusters e a variância dentro de cada cluster;
- esta métrica não tem um intervalo e começa em 0:
  - quanto maior a pontuação, mais densos nossos clusters são;
  - um cluster denso é uma indicação de um cluster bem definido;
- com a biblioteca sklearn podemos facilmente calcular este índice:

```
In [14]: metrics.calinski_harabasz_score(data_points, k_means_model.labels_)

Out[14]: 19.52509172315154
```

Nesse caso, o índice Calinski-Harabasz indica que nossos clusters de modelo k-means estão bastante espalhados e sugere que podemos ter clusters sobrepostos.

Abordagens Intrínsecas (quando não temos as labels)

O Índice Davies-Bouldin:

- mede a similaridade média entre os clusters;
- é a média da semelhança máxima entre cada cluster e o seu par mais similar. A similaridade leva em conta a dispersão interna (distância dos pontos ao centróide) e a distância entre os centróides;
- esta métrica não tem um intervalo e começa em 0:
  - quanto mais próxima a pontuação de 0, melhor; isso significa que os clusters estão bem separados, o que é uma indicação de um bom cluster;
- com a biblioteca sklearn podemos calcular facilmente este índice:

```
In [15]: metrics.davies_bouldin_score(data_points, k_means_model.labels_)

Out[15]: 0.404206621415983
```

Nesse caso, a pontuação Davies-Bouldin indica que nosso algoritmo k-means tem alguns clusters sobrepostos e uma melhoria poderia ser feita separando melhor alguns dos dados de um dos clusters.

Atividade Prática

Atividade Prática

- Faça o download do arquivo “Lab05” disponibilizado em nossa página no moodle.

- **Sugestão:** usar o GoogleColab (https://colab.research.google.com/) para o desenvolvimento e execução dos códigos em python;
- Outras alternativas: Pycharm, Jupyter, Spyder, Visual Studio Code, etc.

Projeto Final

Projeto Final

- Utilizar uma base de dados real, preferencialmente uma que tenha alguma relação com a área de vocês e que vocês detenham conhecimento dos dados;
- Caso não seja possível podem utilizar alguma das bases de dados disponibilizadas em: https://www.kaggle.com/
- Apresentar uma breve descrição da base de dados com as devidas etapas de pré-processamento, quando aplicável (eliminação de dados inválidos, normalização, etc);
- Definir um objetivo de trabalho e aplicar um ou mais métodos de clusterização vistos na disciplina;
- Avaliar os resultados de forma quantitativa (através do cálculo e apresentação de uma ou mais métricas de avaliação de desempenho) e qualitativa (conclusões sobre os agrupamentos);
- Todos as escolhas de projeto devem ser justificadas;
- Entrega de um relatório e o código de implementação da tarefa;
- Data de entrega: **15/05/2026**